

Kinematics Lab

An Interactive Robot Manipulator Simulator
for Education and Code Testing

*A Web-Based Toolkit for Forward/Inverse Kinematics,
Trajectory Planning, Pick-and-Place, and Whiteboard
Drawing with the Universal Robots UR10e*

Jonas Hansen

ME/CDS/EE 235a – Advanced Robotics: Kinematics

California Institute of Technology

March 18, 2026

Built as a self-contained HTML/JavaScript/Three.js application
compatible with any modern web browser on any platform!

Contents

1	Summary	2
1.1	Nature of the Project	2
1.2	Motivation	2
1.3	Scope	2
1.4	Approach	3
2	Technical Details	3
2.1	Denavit–Hartenberg Parameterization	3
2.1.1	UR10e DH Parameters	3
2.2	Forward Kinematics	4
2.2.1	Coordinate System Conversion	4
2.3	Inverse Kinematics for Drawing	5
2.3.1	Tool-Down Constraint	5
2.3.2	Numerical IK Solver	5
2.3.3	Board-to-Local Coordinate Mapping	5
2.3.4	Cartesian Straight-Line Interpolation	6
2.3.5	Aspect Ratio Correction	6
2.4	Collision Detection	6
2.4.1	Floor/Table Collision	6
2.4.2	Self-Collision	7
2.4.3	Object Collision	7
2.5	Object Physics	7
2.6	Python-urx Transpiler	7
2.7	Software Architecture	8
2.8	Gripper Model	8
3	Results and Demonstration	9
3.1	Learn Mode	9
3.2	Play Mode	10
3.3	Simulate Mode	11
3.4	Draw Mode	12
3.5	Video Demonstrations	13
4	Conclusion	13
4.1	Summary of Achievements	13
4.2	Known Limitations	13
4.3	Future Improvements	14
4.4	Closing Remarks	14
A	UR5e and UR3e DH Parameters	15
B	Tool-Down IK Derivation	15

1 Summary

1.1 Nature of the Project

Kinematics Lab (a name I picked because I thought it sounded cool) is a comprehensive, interactive, web-based robot manipulator simulator designed as both an educational tool and a code-testing environment. The simulator accurately models the kinematics of Universal Robots' UR10e, UR5e, and UR3e collaborative robot arms, as well as simplified educational models (2-DOF planar, 3-DOF spatial), using the standard Denavit–Hartenberg (DH) parameterization. The application runs entirely in a single self-contained HTML file—requiring no installation, compilation, or server—and executes in any modern web browser on any operating system.

1.2 Motivation

Robotic manipulation is a cornerstone of modern automation, yet the concepts underlying it—forward kinematics, inverse kinematics, Jacobian-based velocity control, trajectory planning, and collision detection—are inherently spatial and difficult to internalize from textbooks alone. Existing simulators are either expensive commercial products (RoboDK, ROS/Gazebo), require substantial installation effort, or lack the tight integration between theory and interactive visualization that accelerates learning.

This project was motivated by the need for:

- A **zero-friction educational tool** that students can open in a browser and immediately begin exploring.
- A **code-testing sandbox** where users can write and execute robot control code (in both JavaScript and Python-urx syntax) and see the results in real-time 3D.
- A **drawing lab** that connects the abstract mathematics of FK/IK to a tangible task: using a robot arm to draw characters on a whiteboard, exactly as done in a physical robotics laboratory (and what we did for our Lab 2 assignment!).
- Support for the `python-urx` library API, the de facto Python library for controlling Universal Robots hardware, so that code developed in the simulator can transfer directly to real hardware.

1.3 Scope

The simulator encompasses four primary operating modes:

1. **Learn Mode** — 11 interactive lessons covering manipulator fundamentals, DH parameters, homogeneous transforms, FK, IK, velocity kinematics, trajectory planning, grippers, python-urx programming, IK for drawing, and safety.
2. **Play Mode** — Direct joint-angle manipulation via sliders and numeric inputs, with real-time FK computation, transformation matrix display, end-effector tracking, coordinate frame visualization, gripper control, and object pick-and-place.
3. **Simulate Mode** — A code editor supporting JavaScript (continuous `update(t)` functions) and Python-urx syntax (`movej`, `move1`, `gripper` commands) for scripted robot motion with console output.
4. **Draw Mode** — A whiteboard-drawing laboratory where a robot arm holds a pen and draws on a flat surface using either direct joint commands (`rob.movej`) or Cartesian `draw_to(x,y)` commands with automatic IK solving.

1.4 Approach

The simulator was built using:

- **Three.js** (r128) for hardware-accelerated 3D rendering with shadows, physically-based materials, and camera orbit controls.
- **Vanilla JavaScript** for all kinematics computations, IK solving, physics simulation, code parsing, and UI management.
- **Standard DH convention** for all kinematic models, with DH parameters sourced from Universal Robots' official documentation.
- **Damped least-squares (Levenberg–Marquardt)** numerical IK for the drawing mode's Cartesian line interpolation.
- **Simple rigid-body physics** (gravity, bounce, friction) for object interaction after release.
- **Segment-to-segment distance** computation for real-time self-collision, floor collision, and object collision detection.

2 Technical Details

2.1 Denavit–Hartenberg Parameterization

Each joint-link pair in a serial manipulator is described by four scalar DH parameters. The transformation from frame $\{i - 1\}$ to frame $\{i\}$ is:

$$T_i = R_z(\theta_i) \cdot T_z(d_i) \cdot T_x(a_i) \cdot R_x(\alpha_i) \quad (1)$$

Expanding this product yields the standard DH transformation matrix:

$$T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

where θ_i is the joint variable for revolute joints, d_i is the joint variable for prismatic joints, a_i is the link length, and α_i is the link twist.

2.1.1 UR10e DH Parameters

The UR10e's DH table, sourced from Universal Robots' kinematics documentation, is:

Table 1: Standard DH parameters for the UR10e.

Joint	a_i (m)	d_i (m)	α_i (rad)	θ_i offset (rad)
1 (Base)	0	0.1807	$\pi/2$	0
2 (Shoulder)	-0.6127	0	0	0
3 (Elbow)	-0.5716	0	0	0
4 (Wrist 1)	0	0.1742	$\pi/2$	0
5 (Wrist 2)	0	0.1199	$-\pi/2$	0
6 (Wrist 3)	0	0.1166	0	0

Joints 2 and 3 have large negative a values (-0.6127 m, -0.5716 m) corresponding to the upper arm and forearm lengths. Joint 1's d value (0.1807 m) represents the base height, and the $\alpha = \pi/2$ makes the shoulder axis perpendicular to the base rotation axis. Joints 4–6 form a spherical wrist with small d offsets.

2.2 Forward Kinematics

The forward kinematics (FK) of an n -DOF serial manipulator computes the end-effector pose $T_{0n} \in SE(3)$ from the joint configuration $\mathbf{q} = (\theta_1, \dots, \theta_n)$:

$$T_{0n} = \prod_{i=1}^n T_i(\theta_i) = T_1 \cdot T_2 \cdots T_n \quad (3)$$

The end-effector position is extracted as:

$$\mathbf{p} = \begin{bmatrix} T_{0n}[0][3] \\ T_{0n}[1][3] \\ T_{0n}[2][3] \end{bmatrix} \quad (4)$$

and the orientation is the upper-left 3×3 rotation submatrix $R = T_{0n}[0:3, 0:3]$.

Algorithm 1 Forward Kinematics Computation

Require: DH parameters $\{a_i, d_i, \alpha_i\}_{i=1}^n$, joint angles $\{\theta_i\}_{i=1}^n$

Ensure: Transformation matrices $\{T_{0i}\}_{i=0}^n$

- 1: $T_{00} \leftarrow I_{4 \times 4}$
 - 2: **for** $i = 1$ to n **do**
 - 3: Compute T_i from Eq. (2) using $a_i, d_i, \alpha_i, \theta_i$
 - 4: $T_{0i} \leftarrow T_{0,i-1} \cdot T_i$
 - 5: Store T_{0i} for frame visualization and collision detection
 - 6: **end for**
 - 7: **return** $\{T_{0i}\}$
-

This runs in $O(n)$ time with n matrix multiplications—each a 4×4 product requiring 64 multiply-add operations.

2.2.1 Coordinate System Conversion

The DH convention uses axes $(X_{\text{DH}}, Y_{\text{DH}}, Z_{\text{DH}})$ while Three.js uses a Y-up coordinate system. The conversion used throughout the simulator is:

$$\begin{bmatrix} x_{\text{TJS}} \\ y_{\text{TJS}} \\ z_{\text{TJS}} \end{bmatrix} = \begin{bmatrix} X_{\text{DH}} \\ Z_{\text{DH}} \\ -Y_{\text{DH}} \end{bmatrix} \quad (5)$$

In draw mode, the robot group additionally has a rotation $R_y(-\pi/2)$ applied so that the arm (which extends along local $-X$) points toward the whiteboard (world $-Z$ direction). All world-space computations use Three.js's `localToWorld()` to properly account for this group transform.

2.3 Inverse Kinematics for Drawing

2.3.1 Tool-Down Constraint

For whiteboard drawing, the pen must remain perpendicular to the board surface (tool Z-axis pointing straight down in world coordinates). This constrains three of the six joint angles:

$$\theta_4 = -(\theta_2 + \theta_3) - \frac{\pi}{2}, \quad \theta_5 = -\frac{\pi}{2}, \quad \theta_6 = 0 \quad (6)$$

This reduces the IK problem from 6 unknowns to 3: $(\theta_1, \theta_2, \theta_3)$ controlling (x, y, z) of the end-effector.

2.3.2 Numerical IK Solver

With the tool-down constraint, the 3-DOF IK problem is solved using damped least-squares (Levenberg–Marquardt):

$$\Delta\theta = (J^T J + \lambda I)^{-1} J^T \mathbf{e} \quad (7)$$

where $\mathbf{e} = \mathbf{p}_{\text{target}} - \mathbf{p}_{\text{current}}$ is the position error, $J \in \mathbb{R}^{3 \times 3}$ is the numerical Jacobian, and $\lambda = 0.001$ is the damping factor.

Algorithm 2 Damped Least-Squares IK for Tool-Down Drawing

Require: Target position $\mathbf{p}_t$ in robot-local coordinates, initial guess θ_0

Ensure: Joint angles $(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6)$ or failure

```

1:  $\theta \leftarrow \theta_0$  ▷ Warm-start from previous solution
2: for  $k = 1$  to 150 do
3:   Compute  $\theta_4$  from Eq. (6)
4:    $\mathbf{p}_c \leftarrow \text{FK}(\theta_1, \theta_2, \theta_3, \theta_4, -\pi/2, 0).position$ 
5:    $\mathbf{e} \leftarrow \mathbf{p}_t - \mathbf{p}_c$ 
6:   if  $\|\mathbf{e}\| < 5 \text{ mm}$  then
7:     return  $(\theta_1, \theta_2, \theta_3, \theta_4, -90^\circ, 0)$ 
8:   end if
9:   Compute  $J \in \mathbb{R}^{3 \times 3}$  via finite differences ( $\delta = 0.5^\circ$ )
10:   $\Delta\theta \leftarrow (J^T J + 0.001 \cdot I)^{-1} J^T \mathbf{e}$  ▷ Cramer's rule for  $3 \times 3$ 
11:   $\theta \leftarrow \theta + \Delta\theta$ 
12: end for
13: return FAILURE

```

The Jacobian is computed numerically by perturbing each of $(\theta_1, \theta_2, \theta_3)$ by 0.5° and evaluating the FK difference. The 3×3 system is solved using Cramer's rule (determinant-based) rather than a general matrix inverse, which is both efficient and numerically adequate for this well-conditioned problem.

2.3.3 Board-to-Local Coordinate Mapping

Drawing commands `draw_to(u, v)` specify positions in normalized board coordinates $(u, v) \in [0, 1]^2$. These are mapped to the robot's local frame through:

$$\begin{bmatrix} x_{\text{local}} \\ y_{\text{local}} \\ z_{\text{local}} \end{bmatrix} = \begin{bmatrix} w_z^{\text{world}} - R_Z \\ w_Y^{\text{board}} - R_Y + d_{\text{pen}} \\ R_X - w_x^{\text{world}} \end{bmatrix} \quad (8)$$

where $(w_x^{\text{world}}, w_z^{\text{world}})$ are the world-space coordinates of the board point, (R_X, R_Y, R_Z) is the robot group's position, and $d_{\text{pen}} = 0.19$ m is the pen extension from the end-effector (gripper plate + housing + pen body). This accounts for the inverse of the $R_y(-\pi/2)$ group rotation.

2.3.4 Cartesian Straight-Line Interpolation

To ensure visually straight drawing strokes (as opposed to the curved paths produced by joint-space interpolation), each `draw.to` segment is subdivided into $N = 6$ substeps. At each substep, the IK solver is invoked with the linearly interpolated board UV, and the previous solution is used as the warm-start for the next:

$$(u_k, v_k) = (u_{\text{start}}, v_{\text{start}}) + \frac{k}{N} \left((u_{\text{end}}, v_{\text{end}}) - (u_{\text{start}}, v_{\text{start}}) \right), \quad k = 1, \dots, N \quad (9)$$

2.3.5 Aspect Ratio Correction

The whiteboard geometry is $1.05 \text{ m} \times 0.75 \text{ m}$ (aspect ratio 1.4:1). Without correction, equal Δu and Δv produce physically unequal displacements. All drawing templates apply the correction factor:

$$\Delta u_{\text{physical}} = \frac{0.75}{1.05} \cdot \Delta v_{\text{physical}} \approx 0.714 \cdot \Delta v \quad (10)$$

ensuring that a square in UV space renders as a square on the physical board.

2.4 Collision Detection

Three categories of collision are checked every frame:

2.4.1 Floor/Table Collision

All joint positions and the gripper tip are checked against the ground plane (or table surface in draw mode):

$$\text{collision} \iff \exists i \geq 2 : y_i^{\text{world}} < y_{\text{floor}} + \epsilon \quad (11)$$

where $\epsilon = 5$ mm (a parameter I had to play with to ensure that a collision error would arise only when two objects were visually touching each other when moving them together in the simulation environment). Joints 0 and 1 (base) are excluded since they are legitimately at ground level.

2.4.2 Self-Collision

Non-adjacent link segments are tested using the segment-to-segment distance formula. Given segments $\overline{A_1A_2}$ and $\overline{B_1B_2}$:

$$d = \min_{s,t \in [0,1]} \|(A_1 + s(A_2 - A_1)) - (B_1 + t(B_2 - B_1))\| \quad (12)$$

The minimum is found by solving the linear system for (s, t) and clamping to $[0, 1]$. A collision is flagged when $d < 65$ mm for link pairs with index difference ≥ 3 .

2.4.3 Object Collision

For each scene object, the closest point on each link segment to the object center is computed. A collision is flagged when:

$$d_{\text{closest}} < r_{\text{link}} + r_{\text{object}} \quad (13)$$

where $r_{\text{link}} = 22$ mm (link cylinder radius) and r_{object} is the object's bounding radius.

2.5 Object Physics

Released objects are subject to simple rigid-body dynamics:

$$v_y(t + \Delta t) = v_y(t) + g \cdot \Delta t \quad (g = -9.81 \text{ m/s}^2) \quad (14)$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \cdot \Delta t \quad (15)$$

Floor collision applies a coefficient of restitution ($e = 0.3$) and friction ($\mu = 0.9$):

$$v_y \leftarrow -e \cdot v_y, \quad v_x \leftarrow \mu \cdot v_x, \quad v_z \leftarrow \mu \cdot v_z \quad (16)$$

Objects settle when $|v_y| < 0.02$ m/s at floor level. Tumble rotation is applied proportional to horizontal velocity for visual realism.

2.6 Python-urx Transpiler

The simulator parses a subset of the `python-urx` API from text input and converts it to an executable action sequence:

Table 2: Supported python-urx commands and their simulation behavior.

python-urx Command	Simulated Behavior
<code>rob.movej([q1, ..., q6], a, v)</code>	Joint-space interpolation with easing
<code>grip.open_gripper()</code>	Open gripper, release held object with physics
<code>grip.close_gripper()</code>	Close gripper, attach nearest object
<code>sleep(t)</code>	Wait t seconds
<code>pen.down()</code> / <code>pen.up()</code>	Enable/disable drawing on board contact
<code>pen_color("...")</code> / <code>pen_width(n)</code>	Set drawing style
<code>draw_to(u, v)</code>	IK-solved Cartesian line to board coordinate

Joint angles in `movej` are auto-detected as radians (if $|\theta| \leq 2\pi$) or degrees (if $|\theta| > 2\pi$) and converted accordingly.

2.7 Software Architecture

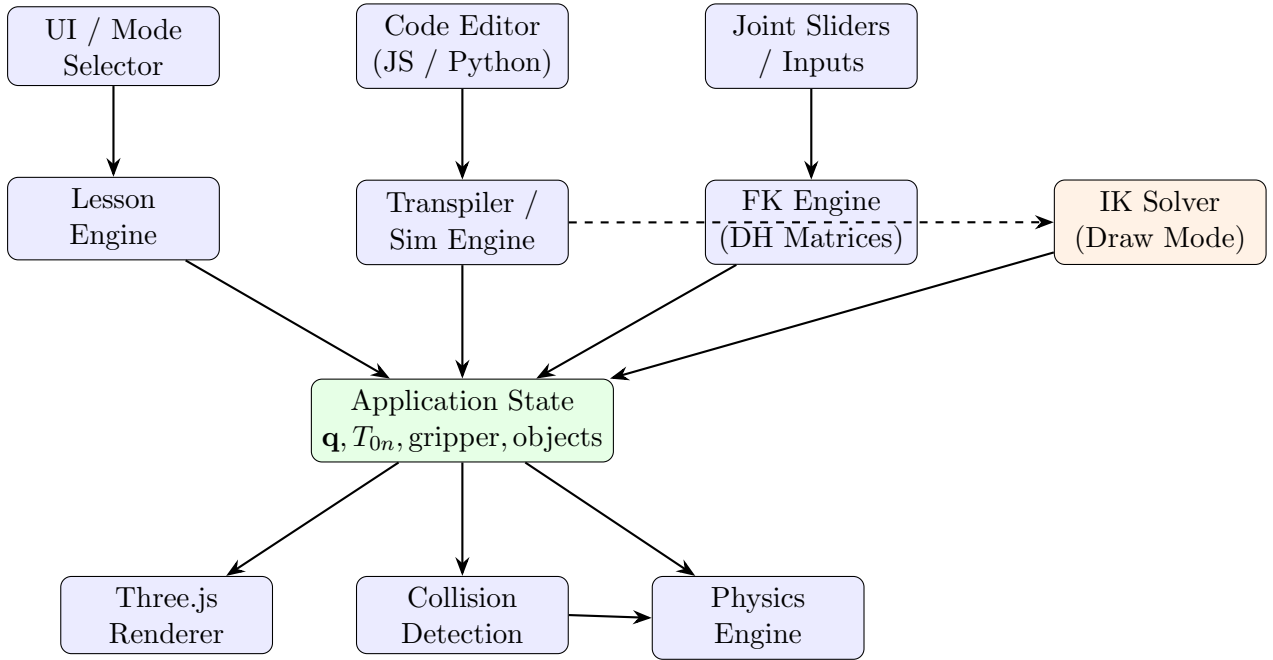


Figure 1: High-level software architecture of the Kinematics Lab simulator.

The application follows a unidirectional data flow: user inputs (mode selection, code, sliders) update the central state object, which triggers a rebuild of the 3D scene. The physics engine runs independently at the animation frame rate. The IK solver is invoked on-demand by the draw engine’s transpiler.

2.8 Gripper Model

The gripper is modeled after the Robotiq 2F fin-ray design:

- **Mounting plate:** cylinder, $r = 30$ mm, $h = 10$ mm
- **Housing:** box, $55 \times 40 \times 35$ mm
- **Fin-ray fingers:** 5 articulated segments per finger with internal rib structures
- **Opening width:** 80 mm (open) / 10 mm (closed)
- **Finger length:** 75 mm with rubber contact tips
- **Pen holder (draw mode):** 120 mm cylindrical pen body with conical tip
- **Total extension from EE:** 145 mm (gripper) / 190 mm (gripper + pen)

Object grasping uses a proximity threshold of 130 mm from the gripper tip to the nearest object center. When the gripper closes, the nearest object within range is attached and moves with the gripper tip. Upon release, the object inherits zero velocity and falls under gravity.

3 Results and Demonstration

3.1 Learn Mode

The Learn mode provides 11 interactive lessons organized into four chapters: Fundamentals, Kinematics, Dynamics/Motion Control, and Programming. Each lesson configures the 3D viewport to illustrate the current concept (e.g., enabling coordinate frames for the DH lesson, enabling trace for FK).

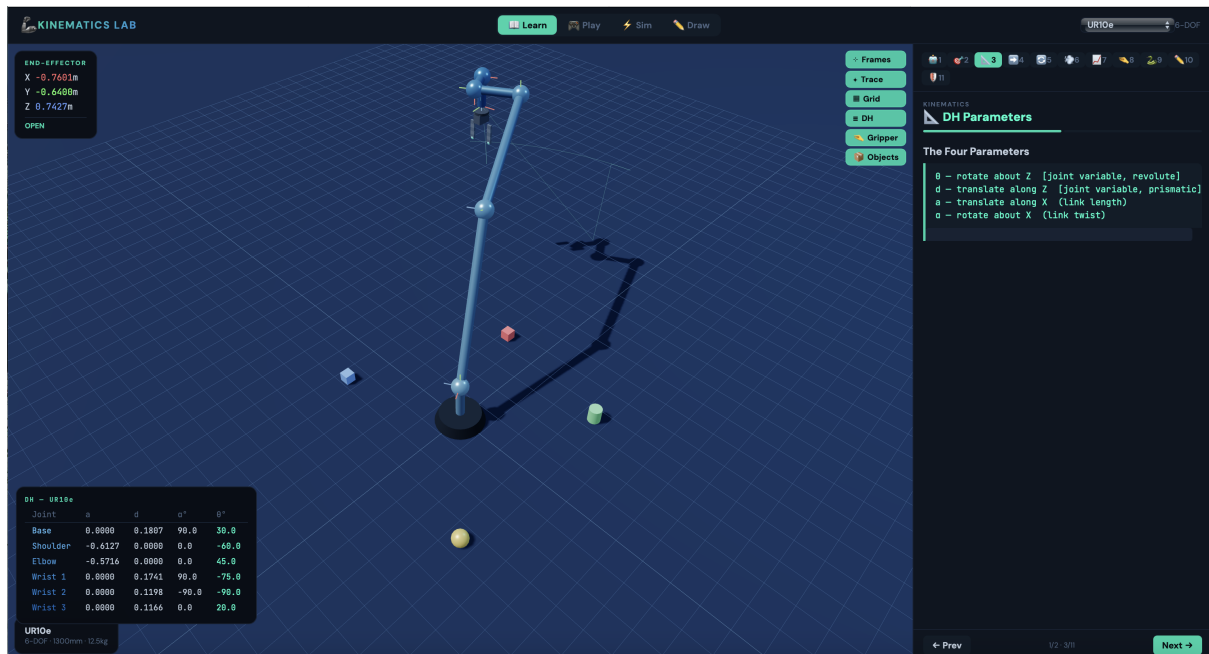


Figure 2: Learn mode — DH Parameters lesson with interactive 3D visualization.

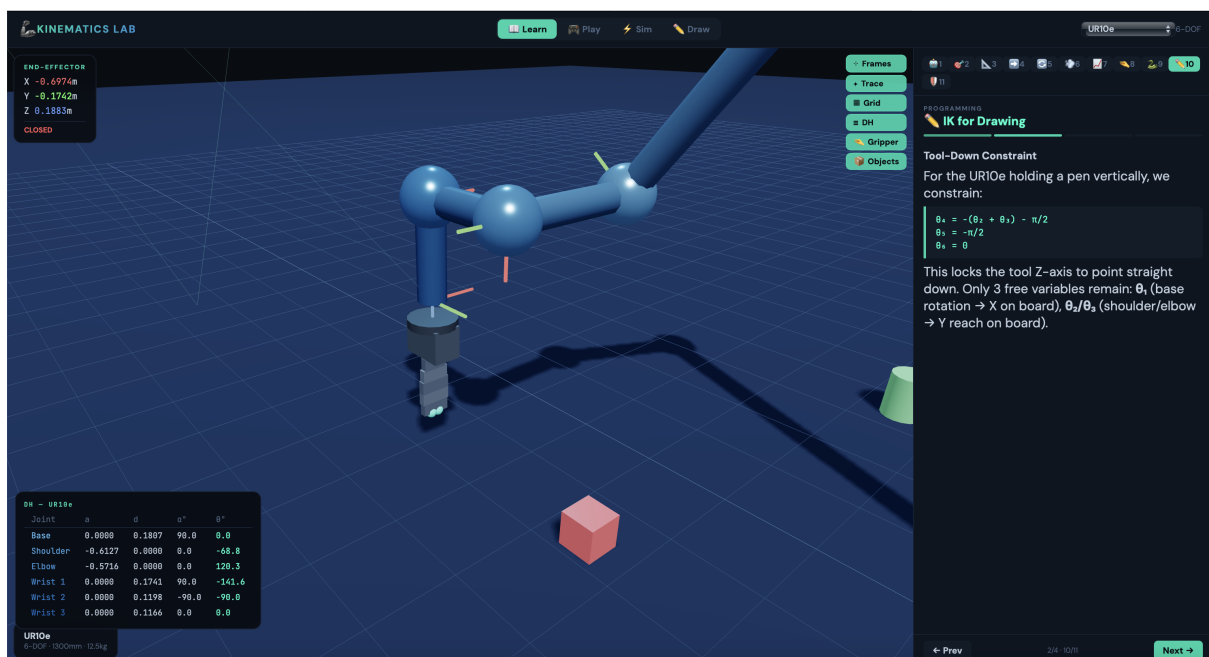


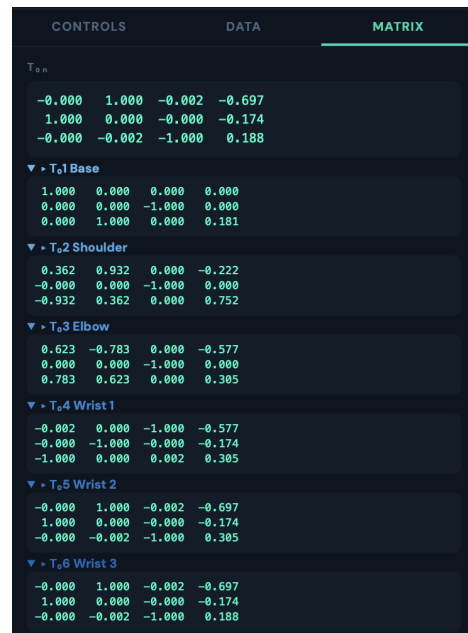
Figure 3: Learn mode — IK for Drawing lesson with tool-down constraint explanation.

3.2 Play Mode

Play mode provides direct manipulation of all joint angles through sliders and numeric inputs, with real-time FK computation.



(a) Controls tab



(b) Matrix tab

Figure 4: Play mode with joint controls and transformation matrix display.

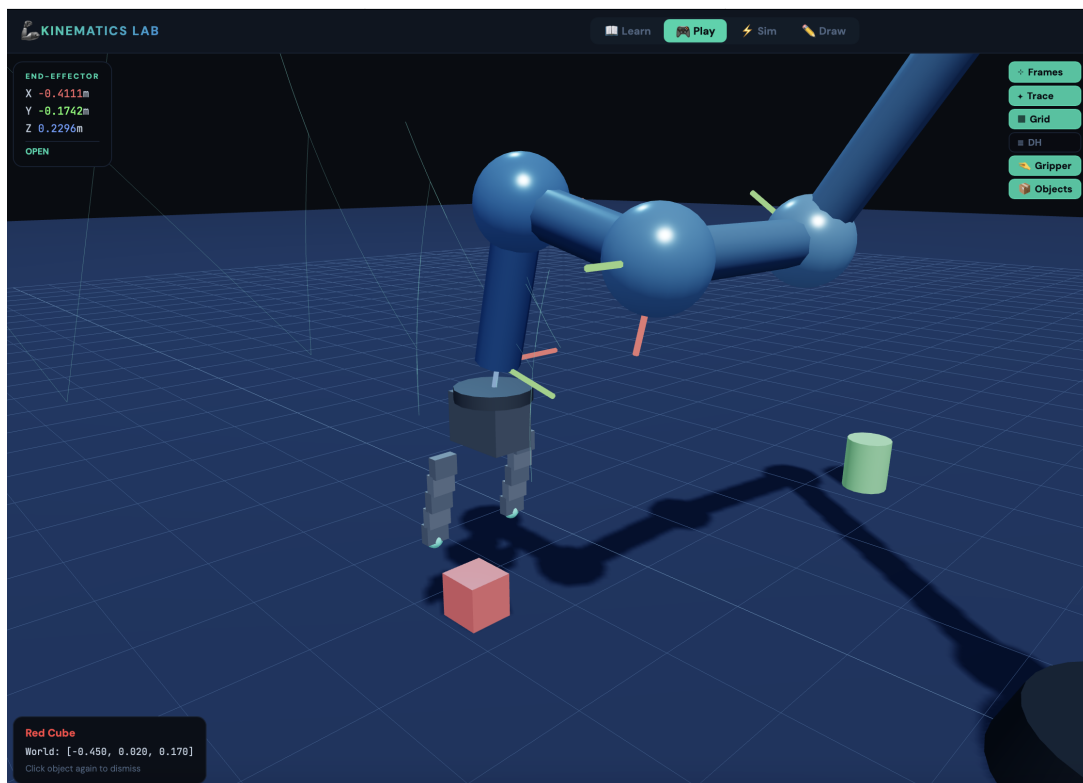


Figure 5: Object coordinate display via click interaction.

3.3 Simulate Mode

The Simulate mode supports both JavaScript (continuous control functions) and Python-urx (sequential command scripts).



Figure 6: Simulate mode with code editor, real-time execution, and console output.

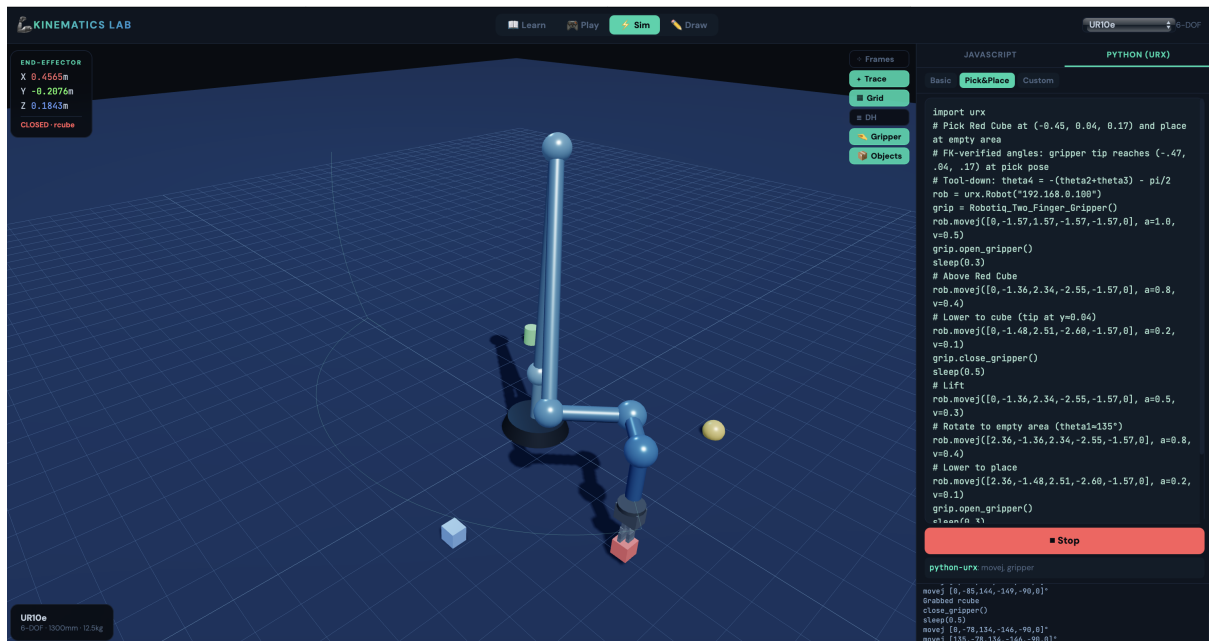


Figure 7: Pick-and-place demonstration with object physics.

3.4 Draw Mode

The Draw mode provides a complete whiteboard-drawing laboratory.

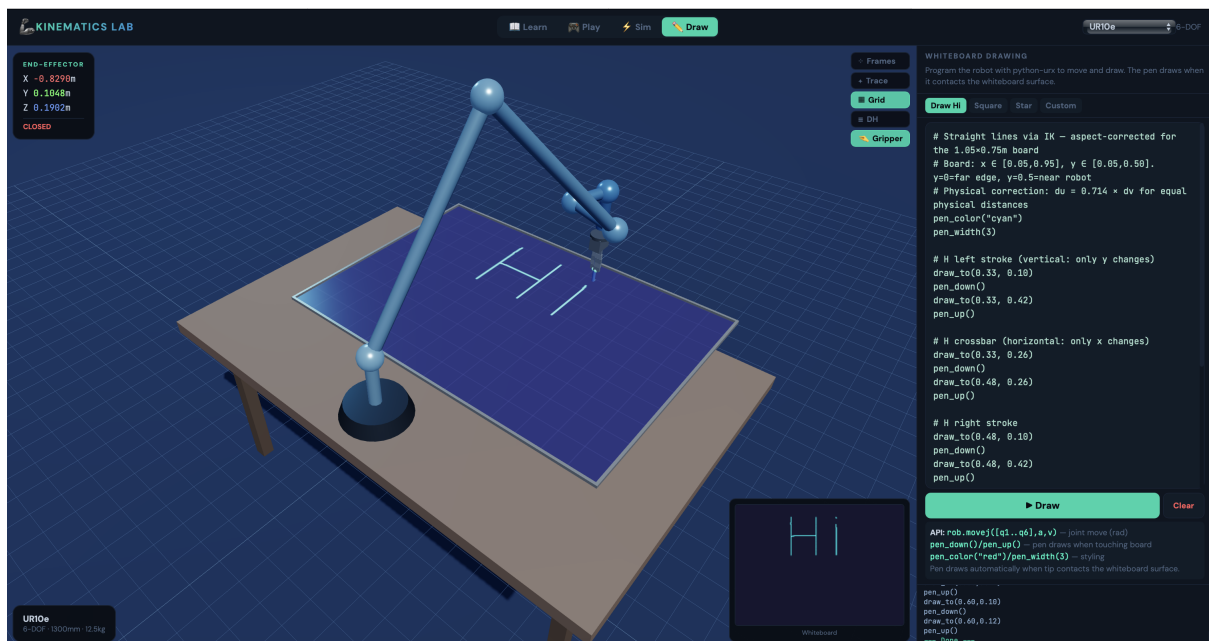
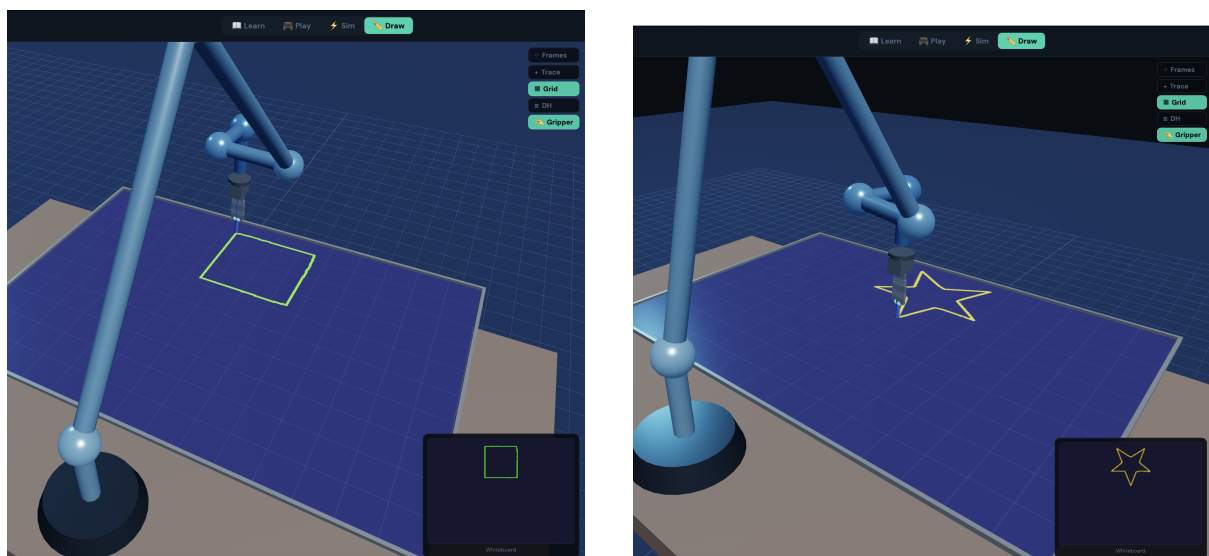


Figure 8: Draw mode with the robot arm drawing “Hi” on the whiteboard. The pen is held by the gripper, perpendicular to the board. The whiteboard preview shows the drawing in progress, and Cartesian-straight lines are solved via numerical IK.



(a) Square

(b) Star

Figure 9: Drawing outputs from the Square and Star built-in templates!

3.5 Video Demonstrations

Table 3: Video demonstrations of the simulator.

Demo	Link
Full simulator walkthrough	https://youtu.be/RkBm2ltJWgI

4 Conclusion

4.1 Summary of Achievements

The Kinematics Lab simulator successfully delivers:

- A **zero-installation**, cross-platform educational tool that runs in any modern browser from a single HTML file.
- Accurate kinematic models of the UR10e, UR5e, UR3e, and educational 2-DOF/3-DOF robots using the standard DH convention.
- Four integrated modes (Learn, Play, Simulate, Draw) covering the full spectrum from passive learning to active code development.
- A functional **python-urx API emulator** that parses and executes `movej`, gripper, and sleep commands with visual feedback.
- A **numerical IK solver** for Cartesian drawing with tool-down constraints, producing visually straight lines on the whiteboard.
- Real-time **collision detection** (self, floor, object) and simple **rigid-body physics** for object interaction.
- **Object interaction** with click-to-inspect coordinates, gripper grasp/release, and gravity-based physics on release.

4.2 Known Limitations

1. **No true dynamics simulation.** Joint torques, inertia, and motor dynamics are not modeled. Motion profiles are kinematic (position-based easing) rather than dynamic.
2. **Simplified collision geometry.** Links are modeled as line segments (not volumes) for collision detection, which can miss edge cases. Object bounding volumes are spherical rather than matching actual geometry.
3. **Limited IK coverage.** The drawing IK solver uses the tool-down constraint, covering only a subset of the workspace. The far half of the whiteboard (high v values) is unreachable, and general 6-DOF IK (arbitrary orientation) is not implemented.
4. **No `move1` (Cartesian linear move) in Simulate mode.** Only `movej` (joint-space) is parsed. Implementing `move1` would require a full 6-DOF IK solver with orientation interpolation.
5. **Joint-space interpolation produces curved paths.** Even in draw mode with `draw_to`, the substep interpolation is joint-space between IK solutions. True Cartesian interpolation would require solving IK at every timestep (computationally intensive in JavaScript).

6. **No velocity/acceleration limits.** Motion speed is controlled by animation duration parameters rather than physically meaningful joint velocity and acceleration bounds.
7. **Single-threaded execution.** The IK solver, physics, rendering, and code parsing all run on the main JavaScript thread. Complex IK sequences or many substeps can cause frame drops.
8. **No persistent state.** The simulator resets when the page is refreshed. There is no save/load for robot configurations, drawing history, or custom code.

4.3 Future Improvements

1. **Full 6-DOF analytical IK** for the UR family, enabling `move1` commands and arbitrary end-effector orientations. The UR10e admits closed-form solutions with up to 8 configurations.
2. **Dynamics simulation** using recursive Newton–Euler or Lagrangian formulation, enabling torque-based control, gravity compensation, and compliance simulation.
3. **ROS/URScript integration** via WebSocket bridge, allowing the simulator to send commands to a real UR robot or a ROS-based simulation (Gazebo, MoveIt).
4. **URDF/mesh loading** for visually accurate robot models with actual CAD geometry rather than sphere-cylinder approximations.
5. **Path planning** with obstacle avoidance (RRT, PRM algorithms) and trajectory optimization.
6. **Multi-robot support** for cooperative manipulation scenarios.
7. **Vision simulation** with a virtual camera mounted on the end-effector for visual servoing demonstrations.
8. **WebGPU acceleration** for more complex scenes and real-time dynamics computation.
9. **Persistent storage** using `localStorage` or IndexedDB for saving configurations, drawings, and custom code.
10. **Collaborative mode** via WebRTC, enabling multiple users to interact with the same robot simultaneously for classroom demonstrations!

4.4 Closing Remarks

The Kinematics Lab demonstrates that a comprehensive robotics educational tool can be built entirely within the browser platform, requiring no specialized software installation. By tightly integrating mathematical theory (DH parameters, FK, IK, Jacobians) with interactive 3D visualization and executable code, the simulator bridges the gap between textbook learning and hands-on laboratory experience. The python-urx API compatibility ensures that skills developed in the simulator transfer directly to real Universal Robots hardware, making it a practical stepping stone for students entering the field of robotic manipulation!

A UR5e and UR3e DH Parameters

Table 4: Standard DH parameters for the UR5e.

Joint	a_i (m)	d_i (m)	α_i (rad)	θ_i offset
1	0	0.1625	$\pi/2$	0
2	-0.4250	0	0	0
3	-0.3922	0	0	0
4	0	0.1333	$\pi/2$	0
5	0	0.0997	$-\pi/2$	0
6	0	0.0996	0	0

Table 5: Standard DH parameters for the UR3e.

Joint	a_i (m)	d_i (m)	α_i (rad)	θ_i offset
1	0	0.15185	$\pi/2$	0
2	-0.24355	0	0	0
3	-0.2132	0	0	0
4	0	0.13105	$\pi/2$	0
5	0	0.08535	$-\pi/2$	0
6	0	0.0921	0	0

B Tool-Down IK Derivation

For the UR10e with a spherical wrist (joints 4–6), the condition for the tool Z -axis to point straight down ($\hat{z}_{\text{tool}} = -\hat{y}_{\text{world}}$) in the DH frame is:

The overall rotation of the tool is:

$$R_{06} = R_z(\theta_1)R'_y(\theta_2 + \theta_3 + \theta_4)R_{\text{wrist}}(\theta_5, \theta_6) \quad (17)$$

For the tool Z to align with the DH $-Z$ axis (which maps to Three.js $-Y$), we need:

$$\theta_2 + \theta_3 + \theta_4 = -\frac{\pi}{2} \quad (18)$$

Solving for θ_4 :

$$\theta_4 = -\theta_2 - \theta_3 - \frac{\pi}{2} \quad (19)$$

With $\theta_5 = -\pi/2$ to align the wrist correctly and $\theta_6 = 0$ (no tool rotation), this yields a pen perfectly perpendicular to any horizontal surface.